

# COMP101

## Introduction to Programming

### 2025-26

## Lecture-27

### 2-d list processing

### Numbers

# List initialise to 1's ('unity')

#function: three parameters with list comprehension

```
def list_init(data, num_rows, num_cols):
```

```
    return [ [data for i in range (num_rows)] for j in range (num_cols) ]
```

```
def main():
```

```
    #call to function: pass three arguments
```

```
    # '1' is value to put in each element, '4' is number of rows, '4' is number of cols
```

```
    unity_matrix = list_init(1, 4, 4)
```

```
    print (unity_matrix)
```

```
main()
```

```
[[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]
```

# array initialise by random

```
import random
```

```
row = 3
```

```
col = 3
```

```
list_3x3 = [
```

```
    [0,0,0],
```

```
    [0,0,0],
```

```
    [0,0,0]
```

```
]
```

```
for r in range(row):
```

```
    for c in range(col):
```

```
        list_3x3[r][c] = random.randint(1, 4)
```

```
print(list_3x3)
```

In every element, put a 0 or 1 or  
2 or 3 or 4 at random

```
[ [2, 4, 4], [1, 2, 3], [3, 4, 1] ]
```

# sum all values in array

```
sum = 0
```

```
list_2x3 = [  
    [1,2,3],  
    [4,5]  
]
```

```
for row in list_2x3:
```

```
    for col in row:
```

```
        sum = sum + col
```

```
[ [1, 2, 3], [4, 5] ]
```

```
print (list_2x3)
```

```
print ("Total of values in array = ",sum)
```

```
Total of values in array = 15
```

# sum row values

```
import random
row = 3
col = 3
list_3x3 = [ [0,0,0], [0,0,0], [0,0,0]]

for r in range(row):
    for c in range(col):
        list_3x3[r][c] = random.randint(1, 4)
```

Populate the list with  
random numbers in  
range 1 to 4

```
print(list_3x3)
[[1, 1, 1], [2, 2, 3], [2, 2, 2]]
sum_row = [sum(i) for i in list_3x3]
print (sum_row)
```

```
[3, 7, 6]
```

← totals for row-1, row-2 and row-3

# sum column values

Information only

```
import random
row = 3
col = 3
list_3x3 = [ [0,0,0], [0,0,0], [0,0,0] ]

for r in range(row):
    for c in range(col):
        list_3x3[r][c] = random.randint(1, 4)
```

```
print(list_3x3)
sum_col = [sum(i) for i in zip(*list_3x3)]
print (sum_row)
```

```
[ [2, 3, 4], [4, 2, 2], [4, 1, 2] ]
[10, 6, 8]
```

[2 3 4] [4 2 2] [4 1 2]

zip the array:

[2 4 4] [3 2 1] [4 2 2]

|   |   |   |
|---|---|---|
| 2 | 3 | 4 |
| 4 | 2 | 2 |
| 4 | 1 | 2 |

---

10 6 8

\*list\_3x3 ensures it iterates through every row

# zip() function

Information only

- Combines ('maps', 'merges') two structures into a single structure – a tuple
- Can speed up merging two data structures with large amounts of data in them
  - Usually for further processing later on
- Structures are usually 'iterables' (list, tuple, string, set, dictionary etc)
- Iterables are accessed with a for loop
  - Individual items can be returned

# zip()

Information only

## #two tuples

```
odds = (1,3,5)
```

```
evens = (2,4,6)
```

```
all = zip(odds, evens)
```

```
print (all)
```

```
<zip object at 0x02BF4AF8>
```

#readable output

```
print (tuple(all))
```

```
((1,2), (3,4), (5,6))
```

## #list and tuple

```
a_list = ['one','two','three','four']
```

```
a_tuple = ('1','2','3','4')
```

```
zip_list = zip(a_list, a_tuple)
```

```
for item in zip_list:
```

```
    print(item)
```

```
('one', '1')
```

```
('two', '2')
```

```
('three', '3')
```

```
('four', '4')
```

for loop for readable output:  
If we remove the for loop  
and leave print(item), we get

```
<zip object at 0x02BE4AA8>
```

# zip ()

Information only

## #tuple

```
actor = ("John Smith", "Andrei Stephens", "Dianne Davies")
alien = ("Smioj", "Stena", "David")
cast = zip(actor,alien)
print (tuple(cast))
(('John Smith', 'Smioj'), ('Andrei Stephens', 'Stena'), ('Dianne Davies', 'David'))
```

## #list

```
actor = ["John Smith", "Andrei Stephens", "Dianne Davies"]
alien = ["Smioj","Stena","David"]
cast = zip(actor,alien)
for item in cast:
    print (item)
    ('John Smith', 'Smioj')
    ('Andrei Stephens', 'Stena')
    ('Dianne Davies', 'David')
```